CASE STUDY · FEATURE DELIVERY

HTK Feature Flags

Bật tính năng theo đúng người, đúng tỷ lệ — với quyết định có thể giải thích, cập nhật không ghi đè và audit đầy đủ.

deterministic rollout | SDK cache + ETag | optimistic versioning | admin / viewer

0,0039 ms
p95 evaluate trên production · 2.000 vòng

14/14
unit test pass trên PGlite

Tác giả: HTK 0396241674
Ngày: 11/11/2026
Live: htk-feature-flags.vercel.app
GitHub: github.com/shirmtry/htk-feature-flags

Số liệu đo trên Vercel production ngày 11/11/2026; phương pháp và giới hạn được nêu ở trang 5.

1. Bài toán

Một đội sản phẩm SaaS muốn phát hành tính năng theo từng nhóm người dùng thay vì deploy lại toàn bộ hệ thống. Họ cần bật một luồng checkout mới cho khoảng một phần ba người dùng, mở báo cáo beta cho vài khách hàng doanh nghiệp, và tắt ngay một banner bảo trì khi sự cố kết thúc. Công cụ hiện tại là một bảng boolean theo tên flag, sửa trực tiếp trong database.

Bốn cách một feature flag thất bại

- **Rollout không ổn định:** dùng random mỗi request khiến cùng một người lúc thấy tính năng, lúc không; bug trở nên không tái hiện được.
- **Không giải thích được quyết định:** người dùng hỏi "vì sao tôi không thấy tính năng?", đội vận hành chỉ có một boolean và không biết flag nào, nhánh nào quyết định.
- **Ghi đè im lặng:** hai admin cùng mở form, người lưu sau xoá thay đổi của người trước mà không ai biết.
- **SDK gọi database mỗi request:** tăng latency và tạo phụ thuộc cứng vào DB; khi DB chậm, toàn bộ app chậm theo.

Yêu cầu đặt ra

Yêu cầu	Tiêu chí kiểm chứng
Targeting đúng và ổn định	Cùng user, cùng flag → cùng bucket qua nhiều lần evaluate và deploy
Giải thích được	Mỗi kết quả trả về `reason` và `bucket`
Không ghi đè	Hai lần PATCH cùng version: lần sau nhận 409
SDK nhẹ	Cache TTL, dedupe request, ETag 304, fallback dữ liệu cũ
Phân quyền	Viewer không sửa được qua UI lẫn API
Audit và đo lường	Mọi write có log; benchmark p95 < 50 ms

Phạm vi

Trong phạm vi: flag boolean, rollout theo phần trăm, target user/tenant, CRUD có version, SDK cache, audit, benchmark, hai vai trò. Ngoài phạm vi: flag đa biến thể (string/JSON), experiment assignment theo metric, streaming invalidation và multi-region replication. Việc chọn phạm vi hẹp giúp hoàn thiện đúng các hành vi khó: tính ổn định của bucket, xung đột ghi và cache.

> **Ràng buộc triển khai:** không có credential cho Railway/Fly.io, nên toàn bộ backend chạy trong Next.js API routes trên Vercel. Hệ quả là persistence dùng PGlite + snapshot Vercel Blob thay vì một Postgres quản lý; trade-off này được phân tích ở trang 5.

2. Giải pháp

Kiến trúc

```
Browser (admin / viewer)
   │  session cookie httpOnly
   ▼
Next.js App Router trên Vercel (Node runtime)
   ├─ /api/auth/*            scrypt + sessions
   ├─ /api/flags             CRUD + zod + optimistic version
   ├─ /api/evaluate          evaluateFlag(flag, context)
   ├─ /api/sdk/flags         snapshot + ETag / 304
   ├─ /api/flags/benchmark   SDK cache + p50/p95
   └─ /api/metrics, audit    số liệu từ DB + request_logs
        │
        ▼
   PGlite (PostgreSQL WASM)
        │ JSON snapshot, ETag HEAD
        ▼
   Vercel Blob (private)
```

UI và API nằm trong cùng một deploy Next.js. Mỗi request API mở PGlite trong lambda, đọc snapshot mới nhất từ Blob khi ETag thay đổi, rồi phục vụ. Sau thao tác ghi, state được ghi lại Blob; local dùng PGlite file-backed tại `./.data/pg`.

Mô hình dữ liệu

Bảng	Cột chính	Vì sao
`users`	email unique, password_hash, `role` ∈ admin/viewer	Phân quyền ở server, không tin UI
`flags`	key unique, enabled, rollout_percentage 0–100, target_users/tenants jsonb, version	Một nguồn sự thật cho targeting và version
`audit_logs`	actor, action, detail jsonb, created_at	Giải thích ai đổi gì, khi nào
`benchmark_runs`	rounds, p50_ms, p95_ms, cache_hits/misses	Đo thật, hiển thị lại trên dashboard
`request_logs`	path, status, duration_ms	Metric request và đếm traffic evaluate/SDK
`rate_limits`	key, window_start, count	Giới hạn endpoint nhạy cảm, không persist

Luồng evaluate

```
evaluateFlag(flag, { userId, tenantId }):
  if !flag.enabled                      -> off,  reason "disabled"
  if userId ∈ flag.target_users         -> on,   reason "user_target"
  if tenantId ∈ flag.target_tenants     -> on,   reason "tenant_target"
  if rollout > 0 and có identity:
      bucket = fnv1a(`${flag.key}:${identity}`) % 100
      -> bucket < rollout ? on : off, reason "rollout"
  else                                  -> off,  reason "default"
```

Hàm evaluate là hàm thuần, không truy cập DB, dùng chung cho API, SDK và benchmark. Kết quả trả về gồm `key`, `value`, `reason`, `bucket` và `version` để giải thích từng quyết định.

3. Giải pháp (tiếp)

Cập nhật không ghi đè

1. Form gửi PATCH kèm `version` đã đọc. **2.** SQL: `update ... where id = $1 and version = $2 ... returning *`.
3. 1 dòng trả về → version tăng 1, ghi audit diff. **4.** 0 dòng → `409 version_conflict` kèm version hiện tại.
5. UI giữ nguyên dữ liệu người dùng đã nhập, tải version mới, cho phép gửi lại.

Đây là optimistic locking, không phải khoá bi quan: người dùng không giữ transaction khi đang điền form. Đổi lại, xung đột hiếm gặp sẽ trả lỗi rõ ràng thay vì âm thầm mất dữ liệu. Unique index trên `key` chặn trùng tên ở tầng database; API kiểm tra trước để trả thông báo thân thiện và vẫn bắt lỗi 23505 làm lớp bảo vệ cuối.

SDK cache và ETag

Cold: loader chạy → snapshot lưu cache + `fetchedAt`.
Hit: trong TTL 30 s → trả cache, tăng `hits`.
Dedupe: nhiều lời gọi đồng thời khi cache trống dùng chung một `inflight` promise.
Lỗi refresh: loader throw sau TTL → trả snapshot cũ với `source: "stale"`.
ETag: `/api/sdk/flags` trả `ETag` theo count + max version; `If-None-Match` khớp → `304`, không truyền body.

Client SDK không tự gọi database. Endpoint snapshot vẫn cần session, nhưng ETag biến các lần poll không đổi thành response rỗng. Trong benchmark, client được warm một lần rồi chạy 2.000 vòng: 2.000 cache hit, 1 miss, 1 load. Con số này chứng minh cache hoạt động, không phải tuyên bố về tốc độ mạng.

Phân quyền và audit

Đăng ký mới tạo role `viewer`; admin được seed sẵn. UI ẩn nút ghi với viewer, nhưng quyết định thật nằm ở API: mọi POST/PATCH/DELETE, audit và benchmark kiểm tra `role === 'admin'` và trả 403. Mỗi thao tác ghi gọi một hàm audit chung, lưu actor, action và diff tóm tắt (ví dụ `rollout: [20, 40]`). Nhờ vậy audit không phụ thuộc việc UI có gọi đúng hay không.

Benchmark có phạm vi rõ ràng

Endpoint benchmark chỉ admin, 3 lần/phút. Nó đọc flags một lần (đo `scanMs`), warm SDK client, rồi chạy N vòng evaluate trên 20 context. Mỗi vòng được đo bằng `performance.now()`; kết quả p50/p95 lưu vào `benchmark_runs` và hiển thị lại. Ngân sách đặt ở 50 ms để có biên an toàn lớn hơn nhiều so với kỳ vọng, và số đo thực tế thấp hơn khoảng bốn bậc.

Các quyết định kỹ thuật chính

- **Một hàm evaluate dùng chung** thay vì logic riêng cho API và SDK — tránh lệch kết quả.
- **Version tường minh trong payload** thay vì ETag cho update — dễ hiểu, dễ test, lỗi 409 rõ ràng.
- **jsonb cho target list** — đủ linh hoạt cho demo, vẫn truy vấn được bằng Postgres.
- **FNV-1a 32-bit trên key + identity** — ổn định, không phụ thuộc thư viện, tách bucket giữa các flag.
- **Snapshot Blob có ETag** — giảm tải đọc và tránh restore lặp lại khi state không đổi.
- **Audit ngay trong transaction nghiệp vụ** — không có cửa sổ "đã sửa nhưng chưa log".

4. Đo lường, trade-off và điểm chưa hoàn thiện

Số liệu thật (production Vercel, 11/11/2026)

Phép đo	Kết quả	Cách đo
Evaluate p50 / p95	**0,0023 ms / 0,0039 ms**	2.000 vòng × 5 flags = 10.000 evaluation, SDK cache warm
Cache	2.000 hit / 1 miss / 1 load	Cùng lần benchmark; TTL 60 s
Đọc flags từ PGlite	0,792 ms	`scanMs` trước vòng lặp
Ngân sách p95	< 50 ms → pass	Biên an toàn theo yêu cầu
Unit test	14/14 pass	`npm test`, PGlite thật
ETag	200 → 304	`If-None-Match` với ETag trả về
Xung đột version	409	PATCH lần hai với version cũ
Viewer ghi	403	POST/PATCH/DELETE/audit/benchmark
Benchmark lần 4/phút	429	Rate limit 3/phút/user
Request latency	p50 77 ms · p95 6.910 ms	10 mẫu, gồm cold start — không đủ để kết luận SLA

Benchmark cố ý chỉ đo phần evaluate sau khi snapshot đã nằm trong cache. Nó không đo network, cold start, hay thời gian ghi Blob. Các con số request latency lấy từ `request_logs` và bị chi phối bởi request đầu tiên khởi tạo PGlite, nên được báo cáo kèm số mẫu thay vì trình bày như SLA.

Trade-off đã chấp nhận

PGlite + Blob thay Postgres quản lý
Ưu: deploy một chạm, SQL thật, migration từ zero, không cần credential dịch vụ ngoài.
Nhược: single-writer, last-write-wins nếu nhiều lambda ghi sát nhau; cold start đọc snapshot.
Khi nào đổi: ghi đồng thời hoặc cần replica — chuyển sang Postgres/Neon.

SDK cache TTL thay push/streaming
Ưu: đơn giản, chịu lỗi, ETag giảm payload, fallback stale khi loader lỗi.
Nhược: flag mới có thể trễ tối đa 30 s ở client đang cache.
Khi nào đổi: cần kill-switch tức thời — thêm SSE/WebSocket hoặc pub/sub.

Rate limit ephemeral thay Redis
Ưu: không thêm hạ tầng, đủ chặn thao tác lặp trong một instance.
Nhược: reset khi cold start, không chia sẻ giữa các instance.
Đây là giới hạn có ý thức, không phải bảo vệ chống abuse cấp production.

Điểm chưa hoàn thiện

- Chưa có version history đầy đủ của flag; audit lưu diff tóm tắt, không lưu snapshot trước/sau toàn bộ.
- Chưa test tải nhiều instance ghi đồng thời; kịch bản last-write-wins mới được phân tích, chưa đo.
- Request latency p95 6,9 s trên 10 mẫu là cold start; chưa đo p95 ổn định sau khi warm.
- Chưa có soft delete/khôi phục flag; xoá là vĩnh viễn và được ghi audit.
- SDK mới test fallback stale ở unit test, chưa có bài đo mất mạng kéo dài trên client thật.

5. Bài học

Bài học lớn nhất là tách *quyết định* khỏi *hạ tầng*. Hàm evaluate thuần không biết database hay HTTP tồn tại; nhờ vậy có thể test 10.000 lần trong vài mili-giây, dùng lại cho API, SDK và benchmark, và giải thích từng kết quả bằng `reason`. Khi logic đã đúng, phần còn lại chỉ là chọn cách lưu và cách cache.

Bài học thứ hai là tính ổn định quan trọng hơn tính ngẫu nhiên. Random theo request tạo ra trải nghiệm không tái hiện được; hash theo key + identity biến rollout thành một tập người dùng xác định, có thể kiểm chứng và giải thích. Đổi tỷ lệ chỉ dịch ngưỡng, không xáo trộn những người đã bật.

Bài học thứ ba là optimistic locking cần được thiết kế ở UI, không chỉ ở SQL. Trả 409 là chưa đủ; form phải giữ dữ liệu người dùng vừa nhập, cập nhật version mới và cho gửi lại. Nếu UI xoá trắng form khi xung đột, người dùng sẽ mất công và tưởng hệ thống lỗi.

Cuối cùng, một benchmark chỉ có nghĩa khi nêu rõ nó đo gì. Con số p95 0,0039 ms rất đẹp, nhưng nó đo evaluate trong bộ nhớ sau khi cache đã warm — không phải độ trễ end-to-end. Báo cáo kèm phạm vi đo, số mẫu và các điểm chưa đo giúp người đọc tin con số hơn, và giúp tôi biết bước tiếp theo là đo tải thật thay vì tối ưu thêm một hàm đã đủ nhanh.

> **Hướng tiếp theo:** chuyển persistence sang Postgres/Neon khi cần nhiều instance ghi; thêm SSE hoặc pub/sub để giảm TTL lan truyền; bổ sung version history và khôi phục flag; đo p95 end-to-end sau cold start với lưu lượng tổng hợp.

HTK Feature Flags · Case study · HTK 0396241674 · 11/11/2026 · Next.js 15, React 19, TypeScript, PGlite, Vercel Blob, Vitest.